

Price Sense AI - Write-Up

Prototype: [paste your video link or repo link here] | Code: [GitHub repo link here]

Price Sense AI helps mid-market retailers answer one question before they discount: "Should I run this promotion?" The user describes a proposed promotion (product, discount, timing) and gets a clear recommendation - Run it / Consider it / Don't run it - backed by projected lift, cannibalization, incremental profit, and risk, plus a chart showing profit at every discount level so they can find the optimal discount.

1. How I decided the features in the POC

I started from the core pain in the brief: retailers run promotions on gut feel and can't tell whether a promotion actually made them money. So I anchored every feature to one job - turning a promo idea into a confident, profit-based decision - and cut anything that didn't serve that job in 24 hours. That gave me four must-have features:

- Simple promotion inputs - product, discount %, and timing. The minimum needed to describe a promotion, kept approachable.
- A single, clear recommendation - Run it / Consider it / Don't run it, in plain language with a one-line reason.
- The four numbers that justify the decision - projected lift, cannibalization, incremental profit, and risk - exactly the questions the brief said retailers cannot answer.
- A "best discount" chart - profit across every discount level, which answers the brief's "was 25% off better than 20%?" and turns the tool into an optimizer.

I deliberately left out logins, multi-product baskets, and real data upload - valuable, but not needed to prove the concept in 24 hours.

2. How I created the product

I built it by partnering with an AI coding assistant, which is the workflow the brief was evaluating. I made the product and economic decisions; the assistant handled implementation; I tested and validated each step. I chose Streamlit (a Python framework that renders a web UI from plain Python) to produce a clean, demo-ready app quickly. I built in small, testable increments - inputs, then the recommendation engine, then results and chart, then polish - running the app after each step and trying edge cases to confirm believable behavior. The intelligence is simulated but grounded in real promotion economics: each product has a price, cost, baseline weekly units, and elasticity, and the engine computes lift, cannibalization, incremental profit, and risk.

3. Technical architecture, and how it changes with scale

Today: a single Streamlit app with three layers - an input layer, a recommendation engine isolated in one `analyze()` function, and a presentation layer. Product data lives in a CATALOGS structure that separates data from logic, which is what makes the product horizontal (a new industry is one data entry, no code change).

At scale, components get upgraded while the architecture stays the same: the mocked elasticity is replaced by an ML model trained on the retailer's real point-of-sale data (and because the engine is isolated, this doesn't touch the UI); the in-code catalog moves to a database; and the app splits into a backend service plus a frontend to serve many customers. The guiding principle: I separated logic, data, and UI, so scaling is swapping components, not a rewrite.

4. Challenges I faced

The main challenge was making the simulated intelligence realistic. My first version recommended "Don't run it" for every promotion because the profit model was too pessimistic. I tuned the assumptions (elasticity and cannibalization) until the tool produced a believable spread: shallow discounts on responsive products are profitable, deep discounts lose money, and a low-elasticity staple like milk shouldn't be promoted at all. A related challenge was scoping - deciding what to fake (the data science) versus what to make genuinely real (the experience and the economic logic).

5. What I would add with more time

- Real data ingestion (upload point-of-sale history and train elasticity on it)
- Side-by-side comparison of several products or discount levels
- Seasonality and external factors in timing
- An automatic discount optimizer surfaced up front
- Accounts, saved history, and post-promo actuals vs. prediction

6. Assumptions I made

- Simulated intelligence is acceptable per the brief; the goal is a sound, believable experience.
- Each product's baseline volume, price, cost, and elasticity are representative placeholders standing in for real historical data.
- Lift responds roughly linearly to discount within the modeled range.
- Cannibalization is modeled as a rising share of incremental volume.
- A single promoted product is evaluated at a time (no multi-item baskets in the POC).